HAW HAMBURG

# Digital Circuits

Exercise 3 – Seven Segment Display

**Group/Team No.: G2T2**

**Date: 9.01.2026**

**Name : AKOTH**

**Name : ROBIN**

**Prename : SHARON**

**Prename : FAIZUL AHMED**

**IM No.: 2756343**

**IM No.: 2723877**

| Lab Exercise 3 | max score | score reached |
|---|---|---|
| P1 LED signal assignment | 4 | |
| | | |
| T0 Create Project | 1 | |
| T1 | | |
| T2 | | |
| T3 | | |
| T4 | | |
| T5 | | |
| T6 Timing Analysis | 5 | |
| | | |
| Sum | 40 | |
| | | |

| | | |
|---|---|---|
| **Report** | **15** | |
| | | |
| **Penalty** | | |
| | | |
| **Score exercise 3** | **55** | |
| | | |
| Total score DIP (E1-E3) | 117 | |
| | | |

# 1. Introduction

The purpose of this laboratory exercise is to design, implement, and verify a complete control system for a 4-digit 7-segment display using the Basys 3 FPGA board. The experiment gradually introduces increasing levels of complexity, starting from static display control and progressing to FSM-based digit selection and counter-driven number generation.

# 2. Preparation

## 2.1. Lab Components and Software

1. Hardware: Basys 3 Development Board featuring a Xilinx Artix-7 FPGA (XC7A35T 1CPG236C).
2. Peripherals: Four-digit common-anode 7-segment display, 16 slide switches, and 5 pushbuttons.
3. Software: Xilinx Vivado Design Suite for synthesis, implementation, and bitstream generation

## 2.2. Segment Bit Pattern Definition

Each decimal digit (0-9) requires a unique combination of illuminated segments. Because of the low-active configuration, segment bit patterns must be defined such that bits corresponding to active segments are set to '0'. During the preparation phase, constant bit patterns were defined in VHDL for each digit as shown below. The decimal point was set to '1' since it was not used in the lab.

```
constant ZERO: std_logic_vector(7 downto 0) := "11000000";
constant ONE  : std_logic_vector(7 downto 0) := "11111001";
constant TWO : std_logic_vector(7 downto 0) := "10100100";
constant THREE: std_logic_vector(7 downto 0) := "10110000";
constant FOUR: std_logic_vector(7 downto 0) := "10011001";
constant FIVE: std_logic_vector(7 downto 0) := "10010010";
constant SIX : std_logic_vector(7 downto 0) := "10000010";
constant SEVEN : std_logic_vector(7 downto 0) := "11111000";
constant EIGHT: std_logic_vector (7 downto 0) := "10000000";
constant NINE : std_logic_vector(7 downto 0) := "10010000";
```

## 2.3. FPGA Pin Assignment and Constraints

A constraint file, *seven_segment.xdc*, was used to map logical VHDL ports to the physical pins of the Basys 3 FPGA board. This includes assignments for the system clock, reset signal, switches, segment LEDs, and digit select lines. Correct pin

mapping was verified using the Basys 3 reference manual to ensure proper electrical connections.

# 3. Lab Session and Results

## 3.1. Task 0: Create New Project

A new Vivado project was created by selecting the Basys 3 board (device XC7A35TCPG236-1). The design file *seven_segment.vhd* and the constraint file *seven_segment.xdc* were added to the project.

## 3.2. Task 1: Static Display Test

The goal was to verify correct hardware connections and segment encoding by displaying a fixed number on a specific digit. The task required us to; Display the digit 9 on the 7-segment display, Activate the second digit from the left (AN2) and Ignore switches, FSMs, and counters. The implementation was:

```
LEDs <= NINE;
DIGIT_select <= "1011";
```

**Observed result**: The digit 9 was displayed at the intended position.

## 3.3. Task 2: Mealy FSM for Digit Selection

This task Introduces a finite state machine (FSM) to select the active digit based on the configuration of switches SW0 and SW1.
In a Mealy FSM, outputs depend directly on the current inputs and state. As a result, changes in the switch inputs immediately affect the digit selection.

```
lamda: process(SW)
begin
   case SW(1 downto 0) is
      when "00" => DIGIT_select <= "1110";
      when "01" => DIGIT_select <= "1101";
      when "10" => DIGIT_select <= "1011";
      when "11" => DIGIT_select <= "0111";
      when others => DIGIT_select <= "1111";
   end case;
end process;
```

**Observations:** When the switches SW0 and SW1 were toggled, the active digit changed immediately without waiting for a clock edge. This behaviour is characteristic of a Mealy FSM, where outputs depend directly on current inputs. The number nine was displayed on the selected digit in real time.

## 3.4. Task 3: Moore FSM with Registered inputs

The FSM was modified to follow the Moore model. This requires registering the switch inputs before using them to control the outputs. We implemented this by; Copying SW0–SW1 into a register, Using the registered value as the FSM state and Ensuring output changes only on clock edges.

```
signal sw_register : std_logic_vector(1 downto 0);
```

```
        lamda : process(sw_register)
          begin
            -- default: all digits OFF
            DIGIT_select <= "1111";

            case sw_register is
              when "00" => DIGIT_select <= "1110"; -- rightmost digit
              when "01" => DIGIT_select <= "1101";
              when "10" => DIGIT_select <= "1011"; -- second from left (AN2)
              when "11" => DIGIT_select <= "0111"; -- leftmost digit
              when others => DIGIT_select <= "1111";
            end case;
          end process;
```

**Observations:** After introducing the switch register, changes in the active digit occurred only on rising clock edges.

## 3.5. Counter 0 to 7

In this task, a numeric counter was introduced to generate the values 0 to 7 automatically. A 28-bit unsigned counter was implemented, and the counter was incremented on every clock cycle to implement the digits 0 to 7.

```
signal cnt28 : unsigned(27 downto 0);
cnt28 <= cnt28 + 1;
```

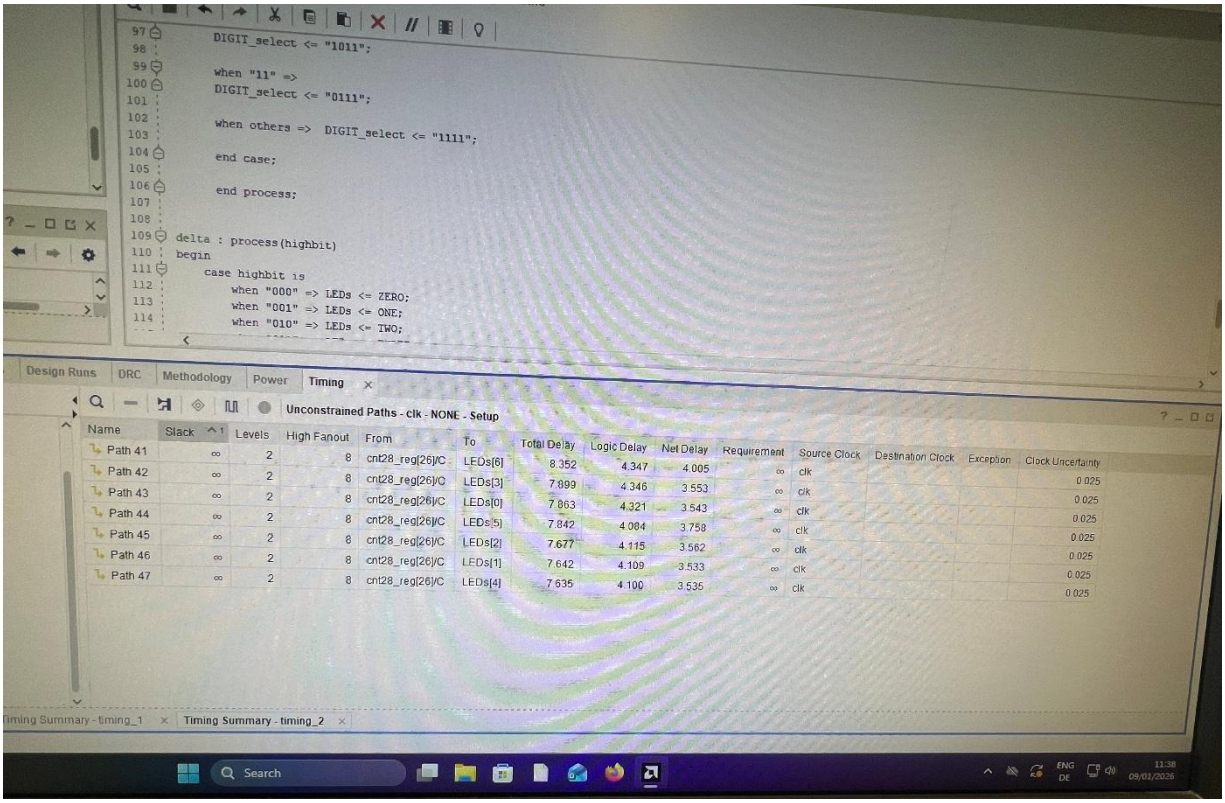
```
highbit <= cnt28(27 downto 25);
```

```
sync : process(clk)
        begin
            if rising_edge(clk) then
                if rst = '1' then
                    cnt28       <= (others => '0');
                    sw_register <= (others => '0');
                else
                    cnt28       <= cnt28 + 1;
                    sw_register <= SW(1 downto 0);
                end if;
            end if;
        end process;
```

```
    delta: process(highbit)
        begin
            case highbit is
                when "000" => LEDs <= ZERO;
                when "001" => LEDs <= ONE;
                when "010" => LEDs <= TWO;
                when "011" => LEDs <= THREE;
                when "100" => LEDs <= FOUR;
                when "101" => LEDs <= FIVE;
                when "110" => LEDs <= SIX;
                when "111" => LEDs <= SEVEN;
                when others => LEDs <= "11111111"; -- all OFF
            end case;
        end process;
```

**Observations:** During operation, the displayed number incremented slowly from zero to seven and then repeated cyclically. The transition speed was visually comfortable and free of flicker, confirming that the use of the most significant counter bits effectively reduced the clock frequency to a human-observable range.

## 3.6. Timing Characteristics




The design was run in Vivado to estimate the speed of the combinational logic. The longest register-to-output path was about **8.352 ns**, which is **4.347 ns** logic delay and **4.005 ns** net delay. This is below the **10 ns** clock period (100MHz); hence the design meets the timing needed.

# 4. Conclusion

In this laboratory exercise, a controller for the Basys 3 seven-segment display was designed using VHDL and successfully tested on FPGA hardware. Initial tests with a static display verified that the segments and digit-select lines operated correctly with low-active logic.

A 28-bit counter was implemented to provide timing, with its most significant bits serving as a state indicator to cycle through the digits in an FSM-like manner. The counter logic was further refined to ensure decimal values looped correctly from 0 to 7.

Observations on the actual hardware confirmed the expected functionality across all features. Timing analysis in Vivado indicated a maximum register-to-output delay of roughly **8.352 ns**, safely below the 10 ns clock period (100 MHz), confirming that the design meets the timing requirements for the chosen clock.

# 5. Full VHDL code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
----------------------------------------
-- Entity
----------------------------------------
entity seven_segment is
   port (
      clk          : in  std_logic;
      rst          : in  std_logic;
      SW           : in  std_logic_vector(7 downto 0);
      LEDs         : out std_logic_vector(7 downto 0);
      DIGIT_select : out std_logic_vector(3 downto 0)
   );
end seven_segment;

-- Architecture
-------------------------------------------
architecture rtl of seven_segment is
-------------------------------------------

   -- 7-segment encoding (low-active)
   constant ZERO  : std_logic_vector(7 downto 0) := "11000000";
   constant ONE   : std_logic_vector(7 downto 0) := "11111001";
   constant TWO   : std_logic_vector(7 downto 0) := "10100100";
   constant THREE : std_logic_vector(7 downto 0) := "10110000";
   constant FOUR  : std_logic_vector(7 downto 0) := "10011001";
   constant FIVE  : std_logic_vector(7 downto 0) := "10010010";
   constant SIX   : std_logic_vector(7 downto 0) := "10000010";
   constant SEVEN : std_logic_vector(7 downto 0) := "11111000";
   constant EIGHT : std_logic_vector(7 downto 0) := "10000000";
   constant NINE  : std_logic_vector(7 downto 0) := "10010000";

--------------------------------------------------
  -- Moore FSM register (Task 3)
--------------------------------------------------
   signal sw_register : std_logic_vector(1 downto 0);

--------------------------------------------------
   -- 28-bit counter (Task 4)
--------------------------------------------------
   signal cnt28   : unsigned(27 downto 0);
   signal highbit : unsigned(2 downto 0);

Begin

   --------------------------------------------------
   -- Sequential process: counter + switch register
   --------------------------------------------------

   sync : process(clk)
   begin
```

```
        if rising_edge(clk) then
            if rst = '1' then
                cnt28       <= (others => '0');
                sw_register <= (others => '0');
            else
                cnt28       <= cnt28 + 1;
                sw_register <= SW(1 downto 0);
            end if;
        end if;
    end process;

    -- Extract highest 3 bits of counter
    highbit <= cnt28(27 downto 25);



-------------------------------------------------------
    -- Moore FSM: Digit selection (Task 3)
-------------------------------------------------------
    lamda: process(sw_register)
    begin
        -- default: all digits OFF
        DIGIT_select <= "1111";

        case sw_register is
            when "00" => DIGIT_select <= "1110";
            when "01" => DIGIT_select <= "1101";
            when "10" => DIGIT_select <= "1011";
            when "11" => DIGIT_select <= "0111";
            when others => DIGIT_select <= "1111";
        end case;
    end process;
-------------------------------------------------
-- TASK 2 (Mealy FSM)
-------------------------------------------------

--lamda: process(SW)
--begin
   -- case SW(1 downto 0) is
     --   when "00" => DIGIT_select <= "1110";
       -- when "01" => DIGIT_select <= "1101";
       -- when "10" => DIGIT_select <= "1011";
       -- when "11" => DIGIT_select <= "0111";
       -- when others => DIGIT_select <= "1111";
  --  end case;
--end process;

-------------------------------------------------------
    -- Display logic: Counter value 0–7 (Task 4)
```

```
    delta: process(highbit)
    begin
        case highbit is
            when "000" => LEDs <= ZERO;
            when "001" => LEDs <= ONE;
            when "010" => LEDs <= TWO;
            when "011" => LEDs <= THREE;
            when "100" => LEDs <= FOUR;
```

```
                when "101" => LEDs <= FIVE;
                when "110" => LEDs <= SIX;
                when "111" => LEDs <= SEVEN;
                when others => LEDs <= "11111111"; -- all OFF
            end case;
        end process;

    end rtl;
```

# 6. Literature


1. Lecture slides: VHDL and FPGA design (IE03-DI).
2. IE03-DI Digital Circuits Laboratory– Exercise 3 documentation, HAW Hamburg.
3. Basys 3 Reference Manual.